RTI ASSIST — BACKEND HANDOVER & TECHNICAL SPECIFICATION REPORT

Target Architecture: Spring Boot 3.x (Java 17/21) + PostgreSQL 15/16 + Spring Security + REST + LLM Client
Author: Antigravity AI (Frontend Architecture Audit)
Date: September 27, 2026
Status: READY FOR BACKEND IMPLEMENTATION (Frontend Complete & Frozen)

TABLE OF CONTENTS

1. Part 1: Full Frontend Audit & Discovery
2. Part 2: Complete Backend API Contract
3. Part 3: Database & Entity Requirements
4. Part 4: LLM Service & AI Prompting Requirements
5. Part 5: Public Authority Directory System
6. Part 6: Case Management & Lifecycle Specification
7. Part 7: Authentication & Access Control (JWT)
8. Part 8: Deadline Division of Responsibility
9. Part 9: File Uploads & Document Storage Policy
10. Part 10: Practical Implementation Checklist (Must vs Later vs Not Needed)
11. Part 11: Boundaries: What to Leave Alone (Frontend Owned)
12. Part 12: Do Not Do Yet — Final Integration Protocol
13. Part 13: Core Architectural Flow Diagrams
14. Part 14: Step-by-Step Backend Development Roadmap
15. Part 15: Backend Environment Variables & Security
16. Part 16: Handover Summaries: What We Need From Each Other
17. Appendix: Executive Summary & Architectural Wrap-Up

PART 1 — FULL FRONTEND AUDIT

The React frontend has been audited in full across all routes, stores, API clients, contexts, components, and unit tests. The frontend application is **100% completed, fully typed, and verified with zero build/lint errors**.

1.1 Complete Route Inventory (`src/App.jsx`)

Route	Component	Access Control	Purpose
`/`	`Home.jsx`	Public	Hero introduction, statutory overview, workflow steps, live metrics radar.
`/intake`	`Intake.jsx`	Public	3-step wizard: Plain language concern → AI RTI Request → Authority recommendation.
`/directory`	`Directory.jsx`	Public	Searchable, filterable directory of public authorities (Central + Delhi NCT focus).

Route	Component	Access Control	Purpose
`/login`	`Login.jsx`	Public	Citizen login via JWT credentials or instant single-click demo account.
`/dashboard`	`Dashboard.jsx`	**Protected**	Operations cockpit with cards/list view, KPI metrics, statutory deadline radar.
`/cases/new/filing`	`CaseFiling.jsx`	**Protected**	Portal link, mandatory instruction block, registration recording form.
`/cases/:id`	`CaseDetail.jsx`	**Protected**	Full case tracking, 4-stage stepper, CPIO roster, queries, editable status, appeal trigger.
`/appeals/:id`	`AppealDraftDetail.jsx`	**Protected**	First/Second appeal generator, editable textarea, copy-to-clipboard, disclaimer.
`*`	`Navigate to="/"`	Public	Catch-all redirect to `/`.

1.2 Frontend Component & State Architecture

- **State Store (`src/lib/store.js`):** Reactive store backed by `localStorage` (`rti_assist_cases`, `rti_assist_appeals`) providing seamless optimistic updates across navigation.
- **Auth Context (`src/context/AuthContext.jsx` & `.tsx`):** Manages `token`, `user`, `login()`, `logout()`, reading from `localStorage.getItem('rti_assist_token')` and `localStorage.getItem('rti_assist_user')`.
- **Protected Route Guard (`src/components/ProtectedRoute.jsx`):** Evaluates `isAuthenticated`. If false, intercepts and redirects to `/login` preserving `state: { from: location }`.
- **Deadline Formatting Engine (`src/lib/deadlineUtils.ts`):** 100% pure client utility computing urgency colors (`red < 3d`, `yellow < 7d`, `green >= 7d`), relative day counts, and labels from a backend-provided `dueDate`. Verified with 27 Vitest unit tests.
- **Navigation (`src/components/Navbar.jsx`):** Global top navigation with route indicators, environment mode badge (`Mock Mode` vs `Live API`), user profile badge, and login/logout CTA.

1.3 Environment Variables

- `VITE_API_URL`: Points to the backend base URL (Default: `http://localhost:8000/api`).
- `VITE_USE_MOCK`: Set to `true` to use client-side `mockApi.js`, or `false` to communicate with the Spring Boot backend.

1.4 How the Frontend Calls the Backend (`src/lib/api.ts` & `src/lib/api.js`)

All HTTP requests go through a unified `request(endpoint, options)` client:
1. Prefixes `VITE_API_URL`.
2. Reads token from `localStorage.getItem('rti_assist_token')`.
3. If token exists, attaches `Authorization: Bearer <token>`.
4. Sets `Content-Type: application/json` and `Accept: application/json`.
5. Unwraps responses:
- If response is `{ success: true, data: { ... } }`, it extracts `json.data`.
- If response is a direct object or array `{ ... }` / `[ ... ]`, it extracts `json` directly.
- **Benefit for Backend:** Spring Boot can return standard DTOs directly or wrap them in a generic `ApiResponse<T>` container.

PART 2 — COMPLETE BACKEND API CONTRACT

The Spring Boot backend must expose the following REST endpoints under the base prefix `/api`.

Endpoint 1: Draft RTI Request (AI LLM)

- **Method & Path:** `POST /api/draft-request`
- **Purpose:** Converts the citizen's unstructured, informal problem into a formal, legally structured RTI application questionnaire under Section 6(1) of the RTI Act, 2005.
- **Authentication:** Public or Optional JWT.
- **Request Headers:** `Content-Type: application/json`
- **Request Body:**

```
{
 "concern": "I applied for the EWS admission quota for my son in DPS Rohini, Delhi 4 months ago. The school claims
}
```

- **Validation Rules:**
- `concern`: String, non-empty, minimum 10 characters, maximum 4,000 characters.
- **Expected Success Response (`200 OK`):**

```
{
 "category": "School Education & Private School Fees",
 "draftText": "APPLICATION FOR OBTAINING INFORMATION UNDER SECTION 6(1) OF THE RIGHT TO INFORMATION ACT, 2005\n\nTo
 "suggestedAuthorityType": "state"
}
```

- **Error Responses:**
- `400 Bad Request`: `{ "code": "VALIDATION_ERROR", "message": "Concern description cannot be blank" }`
- `500 Internal Server Error`: `{ "code": "AI_DRAFT_FAILED", "message": "Upstream LLM generation failed. Please try again." }`

Endpoint 2: Recommend Public Authority (AI Matching)

- **Method & Path:** `POST /api/cases/recommend-authority`
- **Purpose:** Analyzes the formulated RTI request text and matches it to the verified competent Public Authority in the PostgreSQL directory, returning filing channel and fee particulars.
- **Authentication:** Public or Optional JWT.
- **Request Body:**

```
{
 "draftText": "APPLICATION FOR OBTAINING INFORMATION UNDER SECTION 6(1)... Directorate of Education GNCTD..."
}
```

- **Expected Success Response (`200 OK`):**

```
{
 "authority": {
   "id": "auth-5",
   "name": "Directorate of Education, GNCTD",
   "code": "EDUDEL-GNCTD",
   "department": "School Education & Private School Fees",
   "level": "state",
   "jurisdiction": "Delhi NCT",
   "categoryTags": ["School Education", "EWS Admissions", "Fee Regulation"],
   "portalUrl": "https://edudel.nic.in/rti",
   "applicationFee": 10,
   "paymentModes": ["Postal Order", "Online Gateway", "SBI Challan"],
   "pio": {
     "name": "Dr. Rita Sharma",
     "designation": "Additional Director of Education & CPIO",
     "email": "cpio.education@delhi.gov.in",
     "phone": "+91-11-2389-0112",
     "address": "Old Secretariat, Civil Lines, Delhi - 110054"
   },
   "firstAppellateAuthority": {
```

```
      "name": "Shri Himanshu Gupta, IAS",
      "designation": "Director of Education & FAA",
      "email": "diredu@nic.in",
      "address": "Room 12, Old Secretariat, Delhi - 110054"
    }
  },
  "filingChannel": {
    "type": "Online Government Portal (GNCTD)",
    "portalName": "edudel.nic.in",
    "portalUrl": "https://edudel.nic.in/rti",
    "fee": 10,
    "paymentModes": ["UPI", "NetBanking", "Postal Order"]
  },
  "rationale": "Jurisdiction over private recognized schools, EWS admission quotas, and state schooling regulations
  "statutoryTimeframe": "30 calendar days under Section 7(1) of RTI Act, 2005",
  "disclaimer": "This recommendation is an automated administrative suggestion based on keyword patterns and public
}
```

Endpoint 3: List Public Authorities

- **Method & Path:** `GET /api/authorities`
- **Purpose:** Supplies data for the searchable, filterable `/directory` page.
- **Authentication:** Public.
- **Query Parameters:**
- `search` *(optional, string)*: Filter by authority name, ministry, department, category tag, or PIO name.
- `jurisdiction` *(optional, string)*: `'Central'` or `'Delhi NCT'`.
- `level` *(optional, string)*: `'central'`, `'state'`, `'municipal'`, `'statutory'`.
- **Expected Success Response (`200 OK`):**


```
[
  {
    "id": "auth-1",
    "name": "Department of Higher Education",
    "code": "DOHED-HE",
    "ministry": "Ministry of Education",
    "department": "Higher Academic & University Affairs",
    "level": "central",
    "jurisdiction": "Central",
    "categoryTags": ["Universities", "UGC", "Faculty Vacancies", "Grants"],
    "portalUrl": "https://rtionline.gov.in",
    "applicationFee": 10,
    "paymentModes": ["Internet Banking", "UPI", "Credit/Debit Card", "IPO"],
    "pio": {
      "name": "Shri R. K. Sharma",
      "designation": "Director & CPIO",
      "email": "rk.sharma@gov.in",
      "phone": "+91-11-2338-4245",
      "address": "Room 212-C, Shastri Bhawan, New Delhi - 110001"
    },
    "firstAppellateAuthority": {
      "name": "Dr. Anita Karwal",
      "designation": "Joint Secretary & FAA",
      "email": "anita.karwal@gov.in",
      "address": "Shastri Bhawan, New Delhi - 110001"
    }
  }
]
```

Endpoint 4: Get Single Public Authority

- **Method & Path:** `GET /api/authorities/{id}`
- **Purpose:** Retrieves complete profile of a single authority.
- **Authentication:** Public.
- **Path Parameter:** `id` *(string)*.
- **Expected Success Response (`200 OK`):** Returns single `Authority` object.
- **Error Response (`404 Not Found`):** `{ "code": "NOT_FOUND", "message": "Authority not found with id: auth-99" }`

Endpoint 5: Create Case (Record Official Filing)

- **Method & Path:** `POST /api/cases`
- **Purpose:** Saves a registered RTI filing after the citizen has paid and submitted on the official government portal.
- **Authentication: Required (`Bearer <JWT>`)**.
- **Request Body:**

```
{
 "trackingNumber": "DOHED/R/2026/60124",
 "title": "Sanctioned vs Vacant Faculty Posts in Central Universities",
 "subjectCategory": "Higher Academic & University Affairs",
 "summary": "Inquiry into sanctioned, filled, and vacant positions across universities.",
 "description": "Full Section 6(1) text...",
 "queries": [
   "Certified copy of total sanctioned faculty positions.",
   "List of vacant professor positions segregated by reserved categories.",
   "Action taken report on filling vacancies.",
   "Statement of funds allocated for ad-hoc faculty."
 ],
 "applicant": {
   "fullName": "Vikramaditya Roy",
   "email": "vikram.roy@example.com"
 },
 "authorityId": "auth-1",
 "status": "submitted",
 "isLifeAndLiberty": false,
 "filingDate": "2026-09-27T06:30:00.000Z",
 "submissionMode": "online_portal",
 "feePaid": 10,
 "paymentReference": "REG-DOHED/R/2026/60124",
 "documents": [
   {
     "id": "doc-1727418000000",
     "fileName": "portal_acknowledgment.pdf",
     "fileSize": 182400,
     "mimeType": "application/pdf",
     "fileUrl": "#",
     "category": "acknowledgment",
     "uploadedAt": "2026-09-27T06:30:00.000Z"
   }
 ]
}
```

- **Backend Calculation Responsibility:**
- Associate case with the authenticated `user_id` from JWT.
- Compute and insert the default statutory **Deadline**:
  - If `isLifeAndLiberty == true`: `dueDate = filingDate + 48 hours`, `calculatedDays = 2`.
  - If `isLifeAndLiberty == false`: `dueDate = filingDate + 30 days`, `calculatedDays = 30`.
  - `title = "PIO 30-Day Mandatory Response Window"`
  - `type = "pio_response"`
  - `actSectionReference = "RTI Act Sec 7(1)"`
- **Expected Success Response (`201 Created` or `200 OK`):**
  Returns complete `Case` object including generated `id` (e.g. `"case-101"`), `deadlines`, and `createdAt`.

Endpoint 6: List Citizen's Cases

- **Method & Path:** `GET /api/cases`
- **Purpose:** Powers the `/dashboard` page.
- **Authentication: Required (`Bearer <JWT>`)**. Backend must only return cases belonging to the calling user.
- **Query Parameters:**
- `status` *(optional, string)*: Filter by status (`submitted`, `first_appeal_pending`, `pio_replied`, etc.).
- `search` *(optional, string)*: Search by title, tracking number, or authority name.
- `limit` *(optional, integer)*.
- **Expected Success Response (`200 OK`):**
  Returns array of `Case` objects with nested `authority` and `deadlines`.

Endpoint 7: Get Single Case Detail

- **Method & Path:** `GET /api/cases/{id}`
- **Purpose:** Powers `/cases/:id` detail view.
- **Authentication: Required (`Bearer <JWT>`)**.
- **Path Parameter:** `id` *(string)*.
- **Expected Success Response (`200 OK`):** Returns single `Case` object.

Endpoint 8: Update Case (Status & Notes)

- **Method & Path:** `PATCH /api/cases/{id}`
- **Purpose:** Updates case status (e.g. user toggles status to `pio_replied` or logs PIO response summary).
- **Authentication: Required (`Bearer <JWT>`)**.
- **Request Body Example:**

```
{
 "status": "first_appeal_pending"
}
```

Or for logging PIO response:

```
{
 "status": "pio_replied",
 "pioResponseSummary": "PIO furnished partial records; refused item 3 citing Section 8(1)(j).",
 "pioResponseDate": "2026-09-27T08:00:00.000Z"
}
```

- **Expected Success Response (`200 OK`):** Returns updated `Case` object.

Endpoint 9: Draft Statutory Appeal (AI LLM)

- **Method & Path:** `POST /api/draft-appeal`
- **Purpose:** Generates a formal legal petition for either First Appeal (FAA Sec 19(1)) or Second Appeal (CIC Sec 19(3)).
- **Authentication: Required (`Bearer <JWT>`)**.
- **Request Body:**

```
{
 "caseId": "case-101",
 "appealType": "first",
 "caseDetails": null
}
```

Note: `appealType` can be passed as `"first"`, `"first_appeal"`, `"second"`, or `"second_appeal"`.
- **Expected Success Response (`200 OK`):**

```
{
 "draftText": "# FIRST APPEAL UNDER SECTION 19(1) OF THE RIGHT TO INFORMATION ACT, 2005\n\n**BEFORE THE FIRST APPEL
}
```

Endpoint 10: Create Appeal Record

- **Method & Path:** `POST /api/appeals`
- **Purpose:** Saves generated appeal petition into database.
- **Authentication: Required (`Bearer <JWT>`)**.
- **Request Body:** Partial `AppealDraft` object (`caseId`, `type`, `groundOfAppeal`, `draftContent`).
- **Expected Success Response (`201 Created` or `200 OK`):** Returns saved `AppealDraft` object with generated `id`.

Endpoint 11: Authentication (Login)

- **Method & Path:** `POST /api/auth/login`
- **Purpose:** Authenticates citizen and issues signed JWT.
- **Authentication:** Public.
- **Request Body:**

```
{
 "email": "vikram.roy@example.com",
 "password": "password123"
}
```

- **Expected Success Response (`200 OK`):**

```
{
 "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "user": {
   "id": "usr-101",
   "name": "Vikramaditya Roy",
   "email": "vikram.roy@example.com",
   "role": "citizen"
 }
}
```

- **Error Response (`401 Unauthorized`):**

```
{
 "code": "AUTH_FAILED",
 "message": "Invalid email or password"
}
```

PART 3 — DATABASE REQUIREMENTS (POSTGRESQL)

The backend should use a clean, normalized relational model in PostgreSQL.

Classification of Database Entities

Entity	Status	Description
`users`	**REQUIRED**	Citizen account identity, credentials, and ownership foreign keys.
`authorities`	**REQUIRED**	Directory of verified Public Authorities, PIOs, FAAs, and portal URLs.
`cases`	**REQUIRED**	Registered RTI petitions, registration numbers, filing dates, status.
`deadlines`	**REQUIRED**	Statutory tracking milestones, due dates calculated by backend.
`appeals`	**REQUIRED**	First/Second appeal drafts, grounds, and petition texts.
`case_queries`	**OPTIONAL (Recommended)**	Normalized queries list; can alternatively be stored as JSONB in `cases`.
`case_documents`	**OPTIONAL**	Metadata of uploaded receipts (`fileName`, `fileSize`, `uploadedAt`).

Entity	Status	Description
`file_blob_storage`	**NOT CURRENTLY NEEDED**	Binary PDF storage is not used in V1; metadata only.

Entity 1: `users` Table

```
CREATE TABLE users (
   id VARCHAR(64) PRIMARY KEY,              -- e.g. 'usr-101' or UUID
   name VARCHAR(255) NOT NULL,
   email VARCHAR(255) UNIQUE NOT NULL,
   password_hash VARCHAR(255) NOT NULL,     -- BCrypt
   role VARCHAR(32) DEFAULT 'citizen',
   phone VARCHAR(32),
   address TEXT,
   created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

Entity 2: `authorities` Table

```
CREATE TABLE authorities (
   id VARCHAR(64) PRIMARY KEY,              -- e.g. 'auth-1'
   name VARCHAR(255) NOT NULL,
   code VARCHAR(64),                        -- e.g. 'DOHED-HE'
   ministry VARCHAR(255),
   department VARCHAR(255),
   level VARCHAR(32) NOT NULL,              -- 'central', 'state', 'municipal', 'statutory'
   jurisdiction VARCHAR(64) NOT NULL,       -- 'Central', 'Delhi NCT'
   category_tags TEXT[],                    -- e.g. ARRAY['Universities', 'UGC']
   portal_url TEXT,                         -- e.g. 'https://rtionline.gov.in'
   application_fee NUMERIC(6,2) DEFAULT 10.00,
   payment_modes TEXT[],                    -- e.g. ARRAY['UPI', 'Internet Banking']

   -- CPIO Particulars
   pio_name VARCHAR(255) NOT NULL,
   pio_designation VARCHAR(255) NOT NULL,
   pio_email VARCHAR(255) NOT NULL,
   pio_phone VARCHAR(64),
   pio_address TEXT NOT NULL,

   -- First Appellate Authority Particulars
   faa_name VARCHAR(255),
   faa_designation VARCHAR(255),
   faa_email VARCHAR(255),
   faa_phone VARCHAR(64),
   faa_address TEXT,

   created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

Entity 3: `cases` Table

```
CREATE TABLE cases (
   id VARCHAR(64) PRIMARY KEY,              -- e.g. 'case-101'
   user_id VARCHAR(64) NOT NULL REFERENCES users(id),
   tracking_number VARCHAR(128) NOT NULL,   -- Government registration number
   title VARCHAR(255) NOT NULL,
   subject_category VARCHAR(128),
   summary TEXT,
   description TEXT,
   queries JSONB,                           -- Array of strings ["query 1", "query 2"]
   authority_id VARCHAR(64) NOT NULL REFERENCES authorities(id),
   status VARCHAR(32) NOT NULL DEFAULT 'submitted', -- 'submitted', 'pio_replied', 'first_appeal_pending', etc.
   is_life_and_liberty BOOLEAN DEFAULT FALSE,
   filing_date TIMESTAMP WITH TIME ZONE NOT NULL,
   submission_mode VARCHAR(32) DEFAULT 'online_portal',
```

```
   fee_paid NUMERIC(6,2) DEFAULT 10.00,
   payment_reference VARCHAR(128),
   documents JSONB,                          -- Metadata array of attached slips
   pio_response_summary TEXT,
   pio_response_date TIMESTAMP WITH TIME ZONE,
   created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

Entity 4: `deadlines` Table

```
CREATE TABLE deadlines (
   id VARCHAR(64) PRIMARY KEY,               -- e.g. 'dl-1'
   case_id VARCHAR(64) NOT NULL REFERENCES cases(id) ON DELETE CASCADE,
   title VARCHAR(255) NOT NULL,
   type VARCHAR(64) NOT NULL,                -- 'pio_response', 'first_appeal_window', etc.
   due_date TIMESTAMP WITH TIME ZONE NOT NULL,
   calculated_days INTEGER NOT NULL,         -- 30 or 2
   status VARCHAR(32) DEFAULT 'pending',     -- 'pending', 'due_soon', 'overdue', 'completed'
   act_section_reference VARCHAR(64),        -- 'RTI Act Sec 7(1)'
   is_life_and_liberty BOOLEAN DEFAULT FALSE,
   completed_at TIMESTAMP WITH TIME ZONE,
   notes TEXT,
   created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

Entity 5: `appeals` Table

```
CREATE TABLE appeals (
   id VARCHAR(64) PRIMARY KEY,               -- e.g. 'appeal-201'
   case_id VARCHAR(64) NOT NULL REFERENCES cases(id) ON DELETE CASCADE,
   user_id VARCHAR(64) NOT NULL REFERENCES users(id),
   type VARCHAR(32) NOT NULL,                -- 'first_appeal', 'second_appeal'
   appeal_number VARCHAR(128),               -- e.g. 'FAA-2026-DHE-0089'
   authority_id VARCHAR(64) REFERENCES authorities(id),
   ground_of_appeal VARCHAR(64) NOT NULL,    -- 'no_response', 'malafide_denial', etc.
   grounds_summary TEXT NOT NULL,
   detailed_grounds JSONB,
   relief_sought TEXT,
   draft_content TEXT NOT NULL,
   status VARCHAR(32) DEFAULT 'ready_to_send',
   submission_date TIMESTAMP WITH TIME ZONE,
   created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

PART 4 — LLM REQUIREMENTS

Architecture Principle: Server-Side AI Gateway

```
[React Frontend]
     |  (JSON concern or caseId)
[Spring Boot Backend]
     |  (System prompt + User concern + Server-Side API Key)
[Anthropic Claude / Google Gemini API]
     |  (Structured JSON / Markdown text)
[Spring Boot Backend]
     |  (Clean DTO response)
[React Frontend]
```

- **Critical Security Rule:** The LLM API key (Claude/Gemini) lives strictly in Spring Boot `application.yml` / environment variables. It is **never sent to or accessible by the browser**.
- **No Complex NLP Stack:** No vector database, no LangChain complex agents, no local Python ML tokenizer. A clean HTTP REST call from Spring Boot to Claude 3.5 Sonnet or Gemini 1.5 Pro with structured output prompting is all that is required.

Task 1: RTI Request Formulation (`POST /api/draft-request`)

- **Backend Responsibility:**
- Receive `concern` string from frontend.
- Call LLM with a strict system prompt instructing it to produce formal queries under Section 6(1) of the RTI Act, 2005.
- The LLM must convert subjective rants/complaints ("Why did you do this?") into requests for **certified official records** ("Certified copy of file notings, sanction orders, work registers").
- Parse the LLM output into `{ category, draftText, suggestedAuthorityType }`.
- Return JSON to frontend.

Recommended LLM System Prompt:

```
You are an expert Indian Right to Information Act (RTI Act, 2005) legal draftsman.
The citizen will describe a civic grievance in plain language.
Convert their problem into a formal RTI application under Section 6(1).
Rules:
1. Do not ask questions starting with "Why" or asking for opinions (Section 2(f)).
2. Ask for specific certified copies of files, sanction orders, attendance/work logs, expenditure vouchers, and Act
3. If the concern mentions medical emergency, arrest, or danger to life, flag statutory timeline as 48 HOURS under S
4. Return a valid JSON object matching:
  {
    "category": "String (e.g. Infrastructure & Public Works, Education, Municipal)",
    "suggestedAuthorityType": "central | state | municipal",
    "draftText": "Complete formatted letter text ready to submit..."
  }
```

Task 2: Appeal Drafting (`POST /api/draft-appeal`)

- **Backend Responsibility:**
- Receive `caseId` and `appealType` (`"first"` or `"second"`).
- Fetch `Case` and linked `Authority` records from PostgreSQL.
- Build LLM prompt containing:
  - Case tracking number & filing date
  - Number of days elapsed (calculating Section 7(2) deemed refusal)
  - Target Public Authority name & designated FAA name
  - Formulated Section 6 queries
- Prompt LLM to produce a formal memorandum:
  - If `appealType == 'first'`: Appeal before First Appellate Authority under Section 19(1).
  - If `appealType == 'second'`: Second Appeal Memorandum before Central/State Information Commission under Section 19(3) citing Section 20 penalties.
- Return `{ "draftText": "..." }`.

PART 5 — AUTHORITY SYSTEM (DELHI / CENTRAL FOCUS)

5.1 Initial Geographic Scope

Per project requirements, the initial backend database should preload **Central Authorities and Delhi NCT Public Authorities**.

Preloaded Entities in Existing Frontend Directory:
1. Department of Higher Education (Ministry of Education, Central)
2. National Highways Authority of India (NHAI, Central)
3. Brihanmumbai Municipal Corporation (BMC, Municipal benchmark)
4. Food Safety and Standards Authority of India (FSSAI, Central)
5. Directorate of Education, GNCTD (Delhi NCT)
6. Delhi Jal Board (DJB, Delhi NCT)
7. Railway Board (Ministry of Railways, Central)

5.2 Deterministic Authority Recommendation Strategy

To prevent hallucination (LLM inventing fake phone numbers, non-existent CPIOs, or dead URLs):
1. **Frontend calls `POST /api/cases/recommend-authority` with `draftText`.**
2. **Spring Boot parses draft or asks LLM for category keywords** (e.g. `["school", "admission", "ews", "delhi"]`).
3. **Spring Boot queries PostgreSQL `authorities` table:**
`sql SELECT * FROM authorities WHERE 'Delhi NCT' = ANY(jurisdiction) AND category_tags && ARRAY['School Education', 'EWS Admissions'] LIMIT 1;`
4. **Spring Boot returns the verified authority record from PostgreSQL** along with genuine `portalUrl` and authentic `pio` contact info.

PART 6 — CASE MANAGEMENT & LIFECYCLE

6.1 Statutory Status Progression

Cases progress through the following statuses, supported directly by the frontend UI:

```
draft -> submitted -> response_due -> pio_replied -> first_appeal_pending -> second_appeal_pending -> closed
```

6.2 Key Dates Managed by Backend

- `filingDate`: The exact date the application was registered and paid for by the citizen on the official portal.
- `dueDate` (calculated by backend upon case creation):
- Normal Case: `filingDate + 30 calendar days` (RTI Act Sec 7(1))
- Life & Liberty: `filingDate + 48 hours` (RTI Act Sec 7(1) Proviso)
- `firstAppealDeadline`: `dueDate + 30 calendar days` (RTI Act Sec 19(1))
- `secondAppealDeadline`: `firstAppealDecisionDate + 90 calendar days` (RTI Act Sec 19(3))

PART 7 — AUTHENTICATION & ACCESS CONTROL (JWT)

7.1 Protocol

- **Scheme:** Standard JSON Web Token (JWT) in `Authorization: Bearer <token>` header.
- **Payload Claims Required by Frontend:**
- `sub`: User ID (`id`)
- `email`: User email
- `name`: User full name
- `role`: Role (`citizen`)
- `exp`: Expiration timestamp (Recommend 7 days)

7.2 Access Boundaries

- **Unauthenticated Routes:** `POST /api/auth/login`, `GET /api/authorities`, `GET /api/authorities/{id}`, `POST /api/draft-request`, `POST /api/cases/recommend-authority`.
- **Authenticated Routes:** `POST /api/cases`, `GET /api/cases`, `GET /api/cases/{id}`, `PATCH /api/cases/{id}`, `POST /api/draft-appeal`, `POST /api/appeals`, `GET /api/appeals/{id}`, `GET /api/deadlines`.
- **Security Constraint:** A citizen must **never** be permitted to view or update another citizen's case (`WHERE user_id = :authenticatedUserId`).

PART 8 — DEADLINES: DIVISION OF RESPONSIBILITY

To avoid bugs and duplicate business logic, adhere to this strict boundary:

Feature	Backend Responsibility	Frontend Responsibility
Statutory Due Date Calculation	**Yes.** Add 30 days (or 48h) to `filingDate`. Store in `deadlines.due_date`.	No. Takes backend's `dueDate`.
Day Countdown ("X days left")	No. Local calendar clock differs by timezone.	**Yes.** Handled by `deadlineUtils.ts`.
Urgency Colors (Red/Yellow/Green)	No. Visual presentation token.	**Yes.** Handled by `deadlineUtils.ts`.
Overdue & Due-Today flags	No.	**Yes.** Derived relative to user local midnight.
Statutory Section Labels	Store reference in `act_section_reference`.	Formats in badge.

PART 9 — FILE UPLOADS POLICY

Frontend Audit Finding:

- On `/cases/new/filing`, the user can optionally select an acknowledgment slip (`.pdf`, `.png`, `.jpg`).
- **The frontend currently captures file metadata (`fileName`, `fileSize`, `mimeType`) and submits it as a JSON array inside the `POST /api/cases` request.**
- The frontend does **not** make a `multipart/form-data` call and sets `fileUrl: "#"`.

Decision for Backend Friend:

> **Real binary file storage (AWS S3, MinIO, Azure Blob) is NOT REQUIRED for the first backend version.**
> Store the `documents` metadata array in a PostgreSQL `JSONB` column or child table. Multipart upload can be added in a future update if required.

PART 10 — WHAT TO BUILD CHECKLIST

MUST BUILD NOW (Core Backend Milestone)

- [ ] Spring Boot 3.x project initialized with Maven/Gradle.
- [ ] Dependencies: Spring Web, Spring Data JPA, PostgreSQL Driver, Spring Security, jjwt (JWT), Lombok, Validation.
- [ ] PostgreSQL database connection and Flyway/Liquibase or JPA schema generation.
- [ ] Entities: `User`, `Authority`, `Case`, `Deadline`, `Appeal`.
- [ ] Seed data: Preload verified authorities for Central & Delhi NCT.
- [ ] Authentication REST endpoint: `POST /api/auth/login`.
- [ ] JWT Authentication Filter extracting token from `Authorization: Bearer <token>`.
- [ ] LLM Service connecting to Claude 3.5 Sonnet or Gemini 1.5 Pro using server-side secret key.
- [ ] AI Endpoints: `POST /api/draft-request` and `POST /api/draft-appeal`.
- [ ] Authority Endpoints: `GET /api/authorities`, `GET /api/authorities/{id}`, `POST /api/cases/recommend-authority`.
- [ ] Case Endpoints: `POST /api/cases`, `GET /api/cases`, `GET /api/cases/{id}`, `PATCH /api/cases/{id}`.
- [ ] Deadline auto-generation upon case creation.

- [ ] CORS Filter allowing `http://localhost:5173` (Vite dev server) with methods `GET, POST, PUT, PATCH, DELETE, OPTIONS` and headers `Authorization, Content-Type, Accept`.
- [ ] Global `@RestControllerAdvice` error handler returning `{ "code": "...", "message": "..." }`.

SHOULD BUILD AFTER CORE API WORKS

- [ ] Swagger / OpenAPI documentation (`/swagger-ui.html`).
- [ ] User registration endpoint `POST /api/auth/register`.
- [ ] Expanded authority dataset covering additional states (Maharashtra, Karnataka, etc.).
- [ ] Automated email/SMS reminder worker when deadline is 5 days away.

NOT REQUIRED (Explicitly Excluded from Scope)

- [ ] Custom ML models, custom tokenizers, or TF-IDF pipelines.
- [ ] Vector databases (Pinecone, Weaviate, pgvector).
- [ ] Multi-agent complex frameworks.
- [ ] Binary S3 file storage infrastructure.
- [ ] Government captcha bypassers or scraping bots.

PART 11 — WHAT TO LEAVE ALONE (FRONTEND OWNED)

Do **NOT** modify or redesign the following frontend files. They are complete and tested:
1. **React UI & Tailwind Components (`src/pages/*`, `src/components/*`):** Layout, stepper wizards, cards/list view toggles, color gradients, and icons are complete.
2. **Deadline Formatting (`src/lib/deadlineUtils.ts`):** Urgency color calculations (`red < 3d`, `yellow < 7d`, `green >= 7d`) and day counts belong 100% to the frontend.
3. **Form Validations:** Frontend already validates registration numbers, dates, and mandatory authority confirmation checkboxes.
4. **Interactive Textarea & Copy Actions:** Frontend manages clipboard copy and manual text editing.

PART 12 — DO NOT DO YET (FINAL INTEGRATION PROTOCOL)

Do **NOT** attempt frontend/backend integration until the backend is fully developed, running on a known port (e.g. `http://localhost:8000`), and passing its own independent integration tests.

Once the Backend is Ready, We Will Execute:

1. In `rti-assist-frontend/.env`:
   `bash VITE_API_URL=http://localhost:8000/api VITE_USE_MOCK=false`
2. Start Spring Boot (`mvn spring-boot:run`).
3. Start React Frontend (`npm run dev`).
4. Test login & capture real JWT.
5. Submit concern in Intake wizard & verify live LLM draft from Spring Boot.
6. Verify authority matching against PostgreSQL.
7. Record filing on `/cases/new/filing` & verify case and deadline rows in PostgreSQL.
8. Verify `/dashboard` and `/cases/:id` display persisted backend data.
9. Verify `/appeals/:id` generates Second Appeal via backend LLM service.

PART 13 — API FLOW DIAGRAMS

Diagram 1: Intake & RTI Request Drafting

```
Citizen Concern
     |
     v
React Frontend
     |  POST /api/draft-request { "concern": "..." }
     v
Spring Boot Controller
     |  System Prompt + Concern + Server Secret Key
     v
LLM (Claude / Gemini)
     |  Structured JSON Output
     v
Spring Boot Service
     |  { "category": "...", "draftText": "...", "suggestedAuthorityType": "..." }
     v
React Frontend (Displays Step 2 Editable RTI Text)
```

Diagram 2: Authority Matching

```
Drafted RTI Text
     |
     v
React Frontend
     |  POST /api/cases/recommend-authority { "draftText": "..." }
     v
Spring Boot Service
     |  Query PostgreSQL: jurisdiction, keywords, category
     v
PostgreSQL Database
     |  Returns verified Authority row (CPIO, FAA, portal link)
     v
Spring Boot Service
     |  { "authority": { ... }, "filingChannel": { ... }, "rationale": "..." }
     v
React Frontend (Displays Step 3 Authority Card & Portal Link)
```

Diagram 3: Case Creation & Deadline Calculation

```
Official Portal Registration # & Filing Date
     |
     v
React Frontend
     |  POST /api/cases (with Bearer JWT)
     v
Spring Boot Controller
     |  1. Extract userId from JWT
     |  2. Insert row into `cases` table
     |  3. Calculate dueDate = filingDate + 30 days
     |  4. Insert row into `deadlines` table
     v
PostgreSQL Database
     |  Returns created Case entity with nested deadlines
     v
React Frontend (Navigates to /cases/:id and updates Dashboard)
```

PART 14 — BACKEND DEVELOPMENT ORDER

Follow this step-by-step development sequence:

1. **Step 1 — Project Initialization:**
   Create Spring Boot 3.x project (`rti-assist-backend`) with Java 17/21. Configure `application.yml` with PostgreSQL datasource and port `8000`.

2. **Step 2 — Security & JWT:**
   Implement `JwtUtil`, `SecurityFilterChain`, `User` entity, and `POST /api/auth/login`.
3. **Step 3 — Public Authorities:**
   Implement `Authority` entity, `AuthorityRepository`, pre-populate seed data (Delhi + Central), and `GET /api/authorities`.
4. **Step 4 — LLM Integration:**
   Implement `LlmService` with Spring `RestClient` or official SDK calling Claude or Gemini. Implement `POST /api/draft-request`.
5. **Step 5 — Authority Recommendation:**
   Implement `POST /api/cases/recommend-authority` matching request text to preloaded database authorities.
6. **Step 6 — Case & Deadline Management:**
   Implement `Case` and `Deadline` entities. Build `POST /api/cases` (calculating 30-day statutory response date) and `GET /api/cases`.
7. **Step 7 — Appeal Drafting:**
   Implement `POST /api/draft-appeal` connecting case context with LLM to generate Section 19(1) or 19(3) memorandums.
8. **Step 8 — CORS & Error Handling:**
   Configure `CorsConfigurationSource` allowing `http://localhost:5173`. Add `@ControllerAdvice` for clean error formatting.
9. **Step 9 — Postman / Swagger Testing:**
   Test every endpoint independently before connecting the frontend.

PART 15 — ENVIRONMENT VARIABLES

Required Spring Boot Configuration (`application.yml` / System Env)

```
server:
 port: 8000

spring:
 datasource:
   url: ${DATABASE_URL:jdbc:postgresql://localhost:5432/rti_assist}
   username: ${DATABASE_USERNAME:postgres}
   password: ${DATABASE_PASSWORD:postgres}
 jpa:
   hibernate:
     ddl-auto: update
   show-sql: false

jwt:
 secret: ${JWT_SECRET:404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970}
 expiration-ms: 604800000 # 7 days

llm:
 provider: ${LLM_PROVIDER:anthropic} # or gemini
 api-key: ${LLM_API_KEY:your_server_side_secret_key_here}
 model: ${LLM_MODEL:claude-3-5-sonnet-20241022}
```

> [!IMPORTANT]
> **Never commit secrets to GitHub.** Use `.env` or system environment variables for `DATABASE_PASSWORD`, `JWT_SECRET`, and `LLM_API_KEY`.

PART 16 — FINAL HANDOVER SUMMARY

WHAT YOU NEED FROM ME (Frontend Developer):

1. **The frozen frontend repository:** `rti-assist-frontend`.
2. **This specification document:** `BACKEND_HANDOVER_REPORT.md`.
3. **The contract confirmation:** Agree to the exact JSON request/response shapes documented in Part 2.

4. **No UI changes expected:** The frontend will not change its contracts or visual presentation while you build the backend.

WHAT I NEED FROM YOU (Backend Developer Once Finished):

1. **Running Backend URL:** (e.g. `http://localhost:8000/api`).
2. **Confirmation of CORS:** Verified that `http://localhost:5173` is allowed.
3. **Seed Credentials:** One working citizen login (e.g. `vikram.roy@example.com` / `password123`) in the database.
4. **Authority Seed Data:** Verification that at least 5-7 verified Delhi/Central authorities are loaded in the database.
5. **LLM Connectivity:** Confirmation that `/draft-request` and `/draft-appeal` return real AI-generated text using your backend API key.

APPENDIX: EXECUTIVE WRAP-UP & CHECKLISTS

A. Executive Summary

RTI Assist is a modern civic-tech web application that guides Indian citizens from unstructured complaints through AI formulation, official portal filing, statutory 30-day deadline monitoring, and First/Second Appeal generation. The frontend is complete, built in React 19 + Tailwind CSS, and contains a complete mock API layer that switches to live Spring Boot endpoints via `VITE_USE_MOCK=false`.

B. Frontend Architecture Discovered

- **Routing:** React Router v7 with 8 distinct routes.
- **Client Library:** Fetch wrapper with `withRetry`, JWT injection, and error unwrapping.
- **Data Shape:** Typed interfaces for `Case`, `Authority`, `Deadline`, `AppealDraft`, and AI payloads.

C. Backend Checklist Summary

- Build 11 core REST endpoints documented in Part 2.
- Preload verified authorities for Central and Delhi NCT in PostgreSQL.
- Server-side LLM call using Claude or Gemini for request and appeal drafting.
- Automatically generate 30-day response deadline upon case registration.

D. Open Decisions Confirmed

- **File Storage:** Binary storage is postponed. Store file metadata in JSONB.
- **Geographic Scope:** Delhi NCT & Central Government only for Version 1.
- **LLM Provider:** Choice of Anthropic Claude 3.5 Sonnet or Google Gemini 1.5 Pro on Spring Boot.

End of Report.